Motivation

Given the following array,

A	8	6	1	2	7	5	4	3

suppose we need to insert n elements into this array. How can we do this efficiently? Our first pass algorithm will do this in simple way: if the array is full and we need to insert a new element, allocate memory for a new of array that is one larger than the previous, then insert. So, if we were to insert 9 into this array,

A	8	6	1	2	7	5	4	3

Allocate a new empty array that is 1 larger

A'									

Copy all of the data from the old array into the new array

A'	8	6	1	2	7	5	4	3	

Insert the new element in the newly created position

A'	8	6	1	2	7	5	4	3	9

Overwrite A with A'

A	8	6	1	2	7	5	4	3	9

To understand how this algorithm runs we shall performed amortized analysis. That is, we shall investigate what is the "average" running time of n insertions. That is, if we let $T(n)$ be the running time of n insertions, then then the **amortized running time** $AT(n)$ is

$$AT(n) = \frac{T(n)}{n}$$

We analyze this problem by creating a table to track the cost of each insertion. There are two contributing factors that affect the running time: the cost of inserting the element into the array and the cost of creating a new array and copying the data. Assume that the cost of creating a new array and copying the data is proportional to the size of the new array[1] and that inserting the element takes $\Theta(1)$ time.

Operation	**# of elements**	**Array size m**	**Insert time**
	8	8	
Insert	9	9	$c + 9c$
Insert	10	10	$c + 10c$
Insert	11	11	$c + 11c$
Insert	12	12	$c + 12c$
Insert	13	13	$c + 13c$
Insert	14	14	$c + 14c$
Insert	15	15	$c + 15c$
Insert	16	16	$c + 16c$
Insert	17	17	$c + 17c$
Insert	18	18	$c + 18c$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Insert	$n + 6$	$n + 6$	$c + (n + 6)c$
Insert	$n + 7$	$n + 7$	$c + (n + 7)c$
Insert	$n + 8$	$n + 8$	$c + (n + 8)c$

Observe that when we total the final column of this table to retrieve the running time, there are n copies of c each corresponding to the insertions. We typically express the total cost of n insertions in the following manner

$$T(n) = (\text{Cost of insertion}) + (\text{Cost of expansion})$$

and since there are n insertions performed, each of which takes c time, we know that

$$\text{Cost of insertion} = cn.$$

To determine the cost of expansion, we must evaluate the sum

$$\text{Cost of expansion} = 9c + 10c + 11c + 12c + \cdots + c(n + 6) + c(n + 7) + c(n + 8)$$

This is an arithmetic sum which we can analyze by first observing

$$9c + 10c + \cdots + c(n + 7) + c(n + 8) = \sum_{i=9}^{n+8} ci = \sum_{i=1}^{n+8} ci - \sum_{i=1}^{8} ci$$

[1] This assumption is based on the fact that in some programming languages when a new data structure is created, each element of that data structure is initialized. Some languages, like C, do not do this by default.

and we can now exploit the formula for an arithmetic sum,

$$\text{Cost of expansion} = \sum_{i=1}^{n+8} ci - \sum_{i=1}^{8} ci = c\frac{(n+8)(n+9)}{2} + c\frac{8 \cdot 9}{2}.$$

Therefore,

$$T(n) = cn + c\frac{(n+8)(n+9)}{2} + c\frac{8 \cdot 9}{2}$$

So, perform n insertions takes $\Theta(n^2)$ time and the **amortized running time** of insertion is

$$AT(n) = \frac{cn^2}{n} = cn$$

Thus, the amortized time of insertion is $\Theta(n)$. Again, this is a naïve notion of average running time by "averaging" the running time of n insertions.

The question now becomes: can we devise an algorithm that does any better than this?

Examples

Example 1. Suppose we begin with an empty array of size 2 and we must insert n elements into this array. What is the cost of performing all of these insertions if we double the size of the array every time that we must reallocate memory? What is the amortized cost?

Insert elements into the array one at a time

A			A	x_1		A	x_1	x_2

When the array becomes full, create a new array of twice the size

A	x_1	x_2			A	x_1	x_2	x_3	x_4

When the array becomes full, create a new array of twice the size

A	x_1	x_2	x_3	x_4				

We repeat this process until we have inserted n elements into the table. The code for a single insertion can be found here,

```
function SmartInsert(x,A[],m,n)
  ▷ Array A is of size m
  ▷ A contains n elements
    if n = m then
        Create new array A_2 of size 2m
        for i = 1 to m do
            A_2[i] = A[i]
        end for
        Replace A with A_2
        m = 2m
    end if
    n = n + 1
    A[n] = x
end function
```

Assuming that inserting elements takes $\Theta(1)$ time and that doubling the table takes time proportional to the size of the new created array. We construct a table to visualize the cost of each insertion (including the expansion),

Operation	# of elements	Array size m	Insert time
	0	2	
Insert	1	2	c
Insert	2	2	c
Insert	3	4	$c + 4c$
Insert	4	4	c
Insert	5	8	$c + 8c$
Insert	6	8	c
Insert	7	8	c
Insert	8	8	c
Insert	9	16	$c + 16c$
Insert	10	16	c
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Insert	$\frac{n}{2} - 1$	$\frac{n}{2}$	c
Insert	$\frac{n}{2}$	$\frac{n}{2}$	c
Insert	$\frac{n}{2} + 1$	n	$c + cn$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Insert	$n - 1$	n	c
Insert	n	n	c

Let $T(n)$ be the time required to perform n insertions. Then,

$$T(n) = (\text{Cost of insertion}) + (\text{Cost of expansion})$$

As we saw before, there are n copies of c corresponding to the n insertions, therefore

$$\text{Cost of insertion} = cn$$

Furthermore, we can express the cost of expansion as a summation

$$\text{Cost of expansion} = 4c + 8c + 16c + \cdots + \frac{cn}{2} + cn.$$

So,

$$\begin{aligned}
T(n) &= cn + \left(4c + 8c + 16c + \cdots + \frac{cn}{2} + cn\right) \\
T(n) &= cn + \left(cn + \frac{cn}{2} + \frac{cn}{4} + \cdots + 8c + 4c\right) \\
T(n) &= cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{8}{n} + \frac{4}{n}\right)
\end{aligned}$$

We can bound this above by an infinite geometric series,

$$\begin{aligned} T(n) = cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{8}{n} + \frac{4}{n}\right) &\leq cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots\right) \\ &\leq cn + cn\left(\frac{1}{1 - \frac{1}{2}}\right) = 3cn \end{aligned}$$

and bound it below by dropping all but the first term of the summation,

$$T(n) = cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{8}{n} + \frac{4}{n}\right) \geq cn + cn(1) = 2cn$$

Therefore, $T(n) \in \Theta(n)$. That is, the time required to perform n insertions is $\Theta(n)$. Furthermore, the amortized running time of n insertions can be bounded using these estimates,

$$AT(n) \leq \frac{3cn}{n} = 3c \qquad\qquad AT(n) \geq \frac{2cn}{n} = 2c.$$

Thus, $AT(n)$ is in $\Theta(1)$. That is, the "average" running time of a single insertion is constant. Due to the doubling scheme presented, most insertions no longer need to allocate any additional memory, but *some insertions* require cn time.

In the hashing unit we discussed the need to keep the load factor of a hash table $\left(\frac{n}{m}\right)$ less than $\frac{1}{2}$. This was essential to maintaining constant expected running time for open address hashing. Below we introduce table doubling into our `HashTable.Insert` method that we previously implemented.

```
1:  function HashTable.Insert(Key K, Data D)
2:      m = HashTable.size
3:      if HashTable.NumElements ≥ m/2 then
4:          Create new hash table HashTable2 of size 2m
5:          for i = 1 to HashTable.size do
6:              if HashTable[i] is not empty then
7:                  K = HashTable[i].Key
8:                  HashTable2.Insert(K, HashTable[i].Data)
9:              end if
10:         end for
11:         Replace HashTable with HashTable2
12:     end if
13:     HashTable.Insert(K,D)
14: end function
```

Notice that we double the size of the table whenever the load factor would exceed $\frac{1}{2}$, not when the table is full! Let $T(n)$ be the running time of inserting n elements into a Hash Table that is initially empty. Assume that the initial size of the Hash Table is 4 and that it is implemented using Open Address Hashing. Further assume that the hash function is always "well-behaved" enough to ensure that all locations are equally likely when hashed. We can express the running time of this method as

$$T(n) = (\text{Cost of insertion}) + (\text{Cost of expansion})$$

Let $T_{in}(i)$ be the cost of inserting the ith element into the Hash Table on line 13. Then, we can express to cost of all n insertions as

$$\text{Cost of insertion} = \sum_{i=1}^{n} T_{in}(i)$$

Using linearity of expectation we have that

$$E(\text{Cost of insertion}) = \sum_{i=1}^{n} ET_{in}(i) = \sum_{i=1}^{n} c = cn$$

Suppose the Hash Table has size m and is expanded to have size $2m$. We shall analyze the code line by line to compute the expected running time of this single expansion; first creating a new Hash Table on line 4 takes time proportional to the size of the new table, so

$$E(\text{Cost of line 4}) = 2cm.$$

We can express the running time of the for loop on lines 5-10 as

$$\text{Cost of lines 5-10} = \sum_{i=1}^{m} c + \sum_{i=1}^{m} \alpha_i T_{in}(s),$$

where $\alpha_i = 1$ when location i is occupied and 0 otherwise[2]. The first summation corresponds to the constant amount of time required to check whether location i is empty and the second summation corresponds to the cost required to insert the element at location i into `HashTable2`. So, the expected running time of lines 5-10 is

$$E(\text{Cost of lines 5-10}) = cm + \sum_{i=1}^{m} \alpha_i E T_{in}(s) = cm + \sum_{i=1}^{m} c\alpha_i = cm + cn = cm + cn$$

Observe that $n = \frac{m}{2}$ whenever lines 5-10 execute, so

$$E(\text{Cost of lines 5-10}) = \frac{3cm}{2}$$

Therefore, the expected cost of expansion of a table of size m to a table of size $2m$ is

$$E(\text{Cost of expansion from } m \text{ to } 2m) = 2cm + \frac{3cm}{2} = \frac{7cm}{2} = 2c'm$$

We have made this strange change of variable to emphasize that the cost is proportional to the size of the new table. So, we know the cost of a single expansion, we need only determine the sizes that the table is expanded to. We shall construct a table similar to our previous tables, but focusing entirely on the act of resizing the table. The only important quantities for this table are therefore the old size, the new size, and the cost of the expansion.

Old size	**New size**	**Cost of expansion**
4	8	$8c'$
8	16	$16c'$
16	32	$32c'$
$\vdots$	$\vdots$	$\vdots$
$\frac{n}{4}$	$\frac{n}{2}$	$\frac{c'n}{2}$
$\frac{n}{2}$	n	$c'n$
n	$2n$	$2c'n$

Therefore, using linearity of expectation,

$$ET(n) = E(\text{Cost of insertion}) + E(\text{Cost of expansion}) = cn + (8c' + 16c' + 32c' + \cdots + c'n + 2c'n)$$

[2]Note that since `HashTable` contains n elements, $\alpha_i = 1$ for n values of i.

Which will we analyze in a manner similar to our previous algorithm,

$$\begin{aligned}
ET(n) = cn + (8c' + 16c' + 32c' + \cdots + c'n + 2c'n) &= cn + 2c'n\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{4}{c'}\right) \\
&\leq cn + 2c'n\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots\right) \\
&= cn + 4c'n = (c + 4c')n \\
&\geq cn
\end{aligned}$$

Therefore, $ET(n) \in \Theta(n)$. Furthermore, the amortized run time of n insertions is in $\Theta(n/n) = \Theta(1)$.

We observe that it is also possible that if we remove elements from an array, we could achieve arrays that are very empty. Following the example demonstrated earlier we introduce the idea of shrinking the array whenever it is less than half full! This seems like it should be a nice compliment to our method of doubling the size of the table whenever it becomes full.

Example 2. Suppose we begin with an array of size n and we must continually remove elements from this array. What is the cost of performing all of these deletions if we halve the size of the array every time the array is less than half full (i.e. $n \leq m/2$)?

This algorithm seems wise, but how does it interact with our doubling algorithm introduced earlier? Consider the following set of operations:

Operation	**# of elements**	**Array size m**	**Time**
	32	32	
Insert	33	64	$c + 64c$
Delete	32	32	$c + 32c$
Insert	33	64	$c + 64c$
Delete	32	32	$c + 32c$
Insert	33	64	$c + 64c$
Delete	32	32	$c + 32c$

Notice how we must resize the table *every* insertion and deletion. This seems like it could be problematic, so we shall devise a different algorithm that can potentially avoid this issue. What if instead of shrinking to half the size whenever the array is half full, we shrink to half size whenever the array is *much less* than half full? This ensures that there should still some empty locations should we need to insert immediately after reducing the size of the array.

Example 3. Suppose we begin with an array of size n and we must continually remove elements from this array. What is the cost of performing all of these deletions if we halve the size of the array every time the array is less than **a quarter** full (i.e. $n \leq m/4$)?

To gain a bit of an understanding of the problem, we shall (as before) create a table to track the cost of the deletions.

Operation	**# of elements**	**Array size m**	**Delete time**
	18	64	
Delete	17	64	c
Delete	16	32	$c + 32c$
Delete	15	32	c
Delete	14	32	c
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Delete	10	32	c
Delete	9	32	c
Delete	8	16	$c + 16c$
Delete	7	16	c
Delete	6	16	c
Delete	5	16	c
Delete	4	8	$c + 8c$
Delete	3	8	c
Delete	2	4	$c + 4c$
Delete	1	2	c

This table does not fully capture the problem, however. Instead, we shall create two tables that *decouple* the deletion process from the resizing process. So, to analyze the (Cost of deletion):

Number of elements	**Cost of deletion**
n	c
$n-1$	c
$n-2$	c
$\vdots$	$\vdots$
3	c
2	c
1	c

Therefore,

$$\text{Cost of deletion} = cn$$

Now, to analyze the cost of shrinking the table:

Old size	New size	Cost of shrinking
n	$\frac{n}{2}$	cn
$\frac{n}{2}$	$\frac{n}{4}$	$\frac{cn}{2}$
$\frac{n}{4}$	$\frac{n}{8}$	$\frac{cn}{4}$
$\vdots$	$\vdots$	$\vdots$
32	16	$32c$
16	8	$16c$
8	4	$8c$
4	2	$4c$

Therefore,

$$\text{Cost of shrinking} = cn + \frac{cn}{2} + \frac{cn}{4} + \cdots + 32c + 16c + 8c + 4c$$

As we did when considering table insertions, we let $T(n)$ be the running time of deleting n elements from a full table of size n. We can express $T(n)$ as

$$T(n) = (\text{Cost of deletion}) + (\text{Cost of shrinking})$$

So,

$$\begin{aligned}
T(n) &= cn + \left(cn + \frac{cn}{2} + \frac{cn}{4} + \cdots + 8c + 4c\right) \\
T(n) &= cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{8}{n} + \frac{4}{n}\right) \\
T(n) &\leq cn + cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots\right) \\
T(n) &\leq cn + 2cn = 3cn \\
T(n) &\geq 2cn
\end{aligned}$$

Thus, $T(n) \in \Theta(n)$ and the amortized running time of n deletions is $\Theta(1)$.

```
function DynamicPop(x,A[],m,n)
    ▷ Array A is of size m
    ▷ A contains n elements
    if n = 0 then
        error: "Empty array"
    end if
    x = A[n]
    n = n − 1
    if n ≤ m/4 and m ≥ 4 then
        Create new array A_2 of size m/2
        for i = 1 to n do
            A_2[i] = A[i]
        end for
        Replace A with A_2
        m = m/2
    end if
    return x
end function
```